Railway Level Crossing Monitor: A SKY130 RTL-to-GDSII ASIC Design

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ELECTRONICS AND COMMUNICATION ENGINEERING

By

T.MURALI 24EG104C35

Department of ECE

Department of Electronics and Communication Engineering

ANURAG UNIVERSITY

SCHOOL OF ENGINEERING

Venkatapur(V), Ghatkesar(M), Medchal-Malkajgiri Dist-500088

2026-2027

ANURAG UNIVERSITY SCHOOL OF ENGINEERING
Venkatapur(V), Ghatkesar(M), Medchal-Malkajgiri Dist-500088



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"Railway Level Crossing Monitor: A SKY130 RTL-to-GDSII ASIC Design"** being submitted by

T.MURALI 24EG104C35

in partial fulfillment for the award of the Degree of Bachelor of Technology in Electronics & Communication Engineering to the Anurag University, Hyderabad is a record of bonafide work carried out under my guidance and supervision. The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Dr. Harikrishna Kamatham,
Professor, HOD, ECE & ECM

External Examiner

CERTIFICATE FROM ORGANIZATION

(Applicable only if the project was carried out at an organization or company. Insert the certificate issued by the organization on this page; otherwise delete this page.)

DECLARATION

We hereby declare that the project report entitled "Railway Level Crossing Monitor: A SKY130 RTL-to-GDSII ASIC Design", submitted to the Department of Electronics and Communication Engineering, Anurag University, Hyderabad, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering, is a record of original work carried out by us under the guidance of Ankit sir , Department of ECE.

The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma. All sources of information used in the preparation of this report have been duly acknowledged.

T.MURALI 24EG104C35

ACKNOWLEDGEMENTS

We express our sincere gratitude to our project guide, Ankit sir, Department of Electronics and Communication Engineering, for the constant guidance, encouragement and valuable suggestions throughout the course of this project.

We are thankful to Dr. Harikrishna Kamatham, Professor and Head of the Department of Electronics and Communication Engineering, for providing the facilities and the academic environment that made this work possible. We also thank the faculty and staff of the department for their support.

We gratefully acknowledge the open-source VLSI community, whose tools and the SkyWater SKY130 open process design kit made it possible to carry a design all the way from RTL to a verified GDSII layout. Finally, we thank our families and friends for their continued motivation and support.

T.MURALI

ABSTRACT

A railway level crossing must carry out a strict safety sequence every time a train approaches: the warning lights come on, the barrier goes down, the train passes, the barrier goes up and the crossing returns to its idle state. If a step happens out of order or takes too long, the crossing is unsafe, and the failure can be catastrophic. This project presents the design, verification and physical implementation of a dedicated digital Railway Level Crossing Monitor, a hardware watchdog that observes the crossing sensors, verifies the sequence in real time and raises a fail-safe alarm on any violation or timeout. The design targets the SkyWater SKY130 130 nm process.

The monitor reads five sensor inputs (train_approach, train_depart, light_on_sensor, barrier_down_sensor and barrier_up_sensor) together with a clock and a synchronous reset, and produces three outputs: fail_safe, sequence_ok and a 3-bit error_code. It is a six-state finite state machine (S_IDLE, S_WARN, S_BARRIER, S_PASSING, S_CLEAR and S_FAULT) supported by one 8-bit timeout counter shared by all timed states and a 3-bit register that latches the reason for a failure. Any out-of-order sensor event or timeout sends the machine to S_FAULT, where fail_safe stays asserted until reset, and the error code identifies the violation. The monitor only watches the crossing; it does not drive the barriers or the lights.

The RTL was verified with a self-checking testbench of 13 test cases (14 checks), all of which passed in Icarus Verilog, and the same checks passed in gate-level simulation. Yosys mapped the design to 139 standard cells (1,300 μm²) containing 17 flip-flops and no latches. Static timing analysis over 16 process-voltage-temperature corners, before and after layout with extracted parasitics, shows positive setup and hold slack in every corner on a 50 ns clock (post-layout worst setup +28.85 ns, worst hold +0.20 ns). The OpenLane flow produced a 100 μm × 100 μm GDSII layout with zero routing, DRC, antenna, LVS and CVC violations in a total run time of 3 min 27 s. The project shows that a small, deterministic, firmware-free safety monitor can be taken from specification to a sign-off-clean layout using a fully open-source toolchain.

Keywords: ASIC, SKY130, OpenLane, finite state machine, railway level crossing, fail-safe monitor, watchdog timer, static timing analysis, GDSII.

TABLE OF CONTENTS

CERTIFICATE i
CERTIFICATE FROM ORGANIZATION ii
DECLARATION iii
ACKNOWLEDGEMENTS iv
ABSTRACT v
TABLE OF CONTENTS vi
LIST OF FIGURES viii
LIST OF TABLES ix
LIST OF ABBREVIATIONS x
LIST OF SYMBOLS xi
CHAPTER 1 INTRODUCTION 1
1.1 INTRODUCTION 1
1.2 PROBLEM STATEMENT 2
1.3 OBJECTIVE 2
1.4 ORGANIZATION OF THE REPORT 2
CHAPTER 2 LITERATURE SURVEY 4
2.1 LEVEL CROSSING SAFETY AND RAILWAY SIGNALLING 4
2.2 SEQUENCE MONITORING AND WATCHDOG TIMERS 4
2.3 FINITE STATE MACHINES IN DIGITAL CONTROL 4
2.4 OPEN-SOURCE ASIC DESIGN FLOW AND SKY130 5
2.5 SUMMARY AND RESEARCH GAP 5
CHAPTER 3 SOFTWARE AND HARDWARE REQUIREMENTS 6
3.1 SOFTWARE REQUIREMENTS 6
3.2 HARDWARE REQUIREMENTS 6
3.3 DESIGN CONSTRAINTS 7
CHAPTER 4 METHODOLOGY 8
4.1 SYSTEM OVERVIEW 8
4.2 BLOCK DIAGRAM 10
4.3 FLOWCHART 11
4.4 ALGORITHM 12
4.5 SYSTEM ARCHITECTURE 13
4.5.1 Finite state machine 13
4.5.2 Sequential and combinational logic 14
4.6 DESIGN AND VERIFICATION FLOW 15
CHAPTER 5 RESULTS AND DISCUSSION 16
5.1 RTL SIMULATION 16
5.2 LOGIC SYNTHESIS 17
5.3 PRE-LAYOUT STATIC TIMING ANALYSIS 18
5.4 GATE-LEVEL SIMULATION 18
5.5 PHYSICAL DESIGN 19
5.6 POST-LAYOUT TIMING AND PRE/POST COMPARISON 22
5.7 PROBLEMS ENCOUNTERED AND SOLUTIONS 23
5.8 COST, USEFULNESS TO SOCIETY AND ENVIRONMENTAL SAFETY 24

5.9 LIMITATIONS 24
CHAPTER 6 CONCLUSION AND FUTURE SCOPE 26
6.1 CONCLUSION 26
6.2 FUTURE SCOPE 26
REFERENCES 27
APPENDICES 28
APPENDIX A: RTL SOURCE CODE 28
APPENDIX B: SDC AND OPENLANE CONFIGURATION 30
APPENDIX C: COMMANDS TO REPRODUCE THE FLOW 31
APPENDIX D: PROJECT DELIVERABLES 31

LIST OF FIGURES

Figure 4.1: Timing of a normal safe sequence and of a fault case (illustrative) 10
Figure 4.2: Block diagram of the Railway Level Crossing Monitor..... 11
Figure 4.3: Flowchart of the monitor operation..... 12
Figure 4.4: State diagram of the monitor..... 14
Figure 5.1: RTL simulation waveform of the 13 test cases 17
Figure 5.2: Floorplan view (pins and tap cells) 20
Figure 5.3: Placement view after global placement and resizing..... 20
Figure 5.4: Routed layout view..... 21
Figure 5.5: Final GDSII layout view 21
Figure 5.6: Setup and hold slack across the 16 PVT corners, pre- and post-layout..... 23

LIST OF TABLES

Table 3.1: Software tools used 6
Table 3.2: Hardware and technology elements 7
Table 4.1: Functional specification 8
Table 4.2: Interface of the monitor 9
Table 4.3: Timeout limits of the timed states 9
Table 4.4: Error codes 9
Table 4.5: Design decisions and trade-offs 15
Table 5.1: RTL test cases and results 16
Table 5.2: Synthesis results 18
Table 5.3: Physical verification results 19
Table 5.4: Summary of timing results 22
Table 5.5: Pre-layout versus post-layout slack at all 16 corners (ns) 22
Table D.1: Project deliverables 31

LIST OF ABBREVIATIONS

Abbreviation	Expansion
ASIC	Application Specific Integrated Circuit
CTS	Clock Tree Synthesis
CVC	Circuit Validity Checker
DEF	Design Exchange Format
DRC	Design Rule Check
FF	Flip-Flop
FSM	Finite State Machine
GDSII	Graphic Data System II (layout stream format)
GLS	Gate-Level Simulation
HDL	Hardware Description Language
LEF	Library Exchange Format
LVS	Layout Versus Schematic
PDK	Process Design Kit
PDN	Power Distribution Network
PVT	Process, Voltage and Temperature
RTL	Register Transfer Level
SDC	Synopsys Design Constraints
SPEF	Standard Parasitic Exchange Format
STA	Static Timing Analysis
TNS	Total Negative Slack
UDP	User-Defined Primitive
VCD	Value Change Dump
WNS	Worst Negative Slack

LIST OF SYMBOLS

Symbol	Meaning
f_clk	Clock frequency (20 MHz)
T_clk	Clock period (50 ns)
LIMIT_WARN	Timeout limit of S_WARN (200 clock cycles)
LIMIT_BARRIER	Timeout limit of S_BARRIER (250 clock cycles)
LIMIT_PASSING	Timeout limit of S_PASSING (255 clock cycles)
LIMIT_CLEAR	Timeout limit of S_CLEAR (200 clock cycles)
counter	8-bit shared timeout counter
error_code	3-bit code that identifies the violation
t_setup	Setup slack (ns)
t_hold	Hold slack (ns)
V_DD	Supply voltage (1.8 V nominal)
µm, µm²	Micrometre and square micrometre (length and area)

CHAPTER 1
INTRODUCTION

1.1 INTRODUCTION

A railway level crossing is a place where a road and a railway line cross at the same level. It is one of the most safety-critical points of a rail network, because a failure there can put road users and a train in the same place at the same time. When a train approaches, the crossing has to carry out a fixed sequence of actions: the warning lights switch on, the barrier is lowered, the train passes, the barrier is raised and the lights are switched off. Each step must happen in the correct order and within a reasonable time. If the lights do not come on, if the barrier gets stuck, or if a sensor reports an event that does not fit the sequence, the crossing is no longer safe and the signalling system must be told immediately.

Level crossing controllers have traditionally relied on relay logic, programmable controllers and human supervision. A monitoring function that is built from a general-purpose microcontroller needs firmware, a boot sequence and often an operating system, all of which add points of failure and a start-up delay. A dedicated digital circuit, on the other hand, reacts within a single clock cycle, has no software that can crash, and behaves deterministically. For a function this small and this important, a custom Application Specific Integrated Circuit (ASIC) is an excellent vehicle for learning the complete digital design flow while producing a genuinely useful device.

This project designs a compact sequential circuit, the Railway Level Crossing Monitor, and takes it through every stage of the digital ASIC flow on the open-source SkyWater SKY130 130 nm process: specification, RTL coding in Verilog, simulation, logic synthesis, static timing analysis, gate-level simulation, floorplanning, placement, clock-tree synthesis, routing, physical verification and parasitic-aware sign-off timing. The monitor is a watcher: it observes the crossing sensors, checks the sequence and the timing, and raises a fail-safe alarm when something is wrong. The result is a verified GDSII layout that fits in a 100 µm × 100 µm die.

1.2 PROBLEM STATEMENT

A level crossing that does not follow its safety sequence exposes road users to the danger of a train. Typical failures are a warning light that never turns on, a barrier that never reaches the down position, a train that is reported as having left before the barrier was down, or a barrier that does not rise again after the train has passed. Relay logic and manual supervision can miss such conditions, and a software-based checker can itself fail or restart at the wrong moment. A hardware monitor is therefore required that (i) checks that the sensor events occur in the correct order, (ii) times every safety-critical stage and raises a fault if a stage takes too long, (iii) latches the fault so that it cannot disappear by itself, (iv) reports the specific reason through an error code for diagnosis, and (v) never reports a healthy sequence and a fault at the same time, and always starts from a safe, idle state after reset.

1.3 OBJECTIVE

The main objective is to design, verify and implement a fail-safe railway level crossing sequence monitor as a digital ASIC and to produce a sign-off-clean layout using an open-source flow. The specific objectives are:

1. To define a functional specification with the safe crossing sequence, a timeout for each stage, out-of-order detection, a latched fault and a 3-bit error code.
2. To implement the specification as a six-state finite state machine with a shared 8-bit timeout counter in synthesizable, macro-free Verilog-2005.
3. To verify the RTL with a self-checking testbench and the synthesized netlist with gate-level simulation.
4. To synthesize the design to SKY130 high-density standard cells and close timing in all 16 PVT corners, before and after layout.
5. To run the OpenLane RTL-to-GDSII flow in a fixed 100 µm × 100 µm die and achieve zero DRC, LVS, antenna, CVC and routing violations.

1.4 ORGANIZATION OF THE REPORT

Chapter 1 introduces the project. Chapter 2 reviews related work and the technologies on which the design rests. Chapter 3 lists the software tools and the hardware (technology and external components) used. Chapter 4 explains the methodology through the block diagram, flowchart, algorithm and system architecture. Chapter 5 presents the results of simulation, synthesis, timing analysis and physical design, and

discusses cost, social usefulness and environmental safety. Chapter 6 concludes the report and outlines the future scope, followed by references and appendices.

CHAPTER 2
LITERATURE SURVEY

2.1 LEVEL CROSSING SAFETY AND RAILWAY SIGNALLING

Level crossing protection is traditionally implemented with track-side train detectors, warning lights, audible alarms and motor-driven barriers that are sequenced by relay logic or by programmable controllers. Feedback sensors on the lights and on the barrier position tell the controller whether each action really took place. Railway signalling is a safety-critical field, and the design of its electronic equipment is governed by functional-safety standards such as IEC 61508 [10] and the railway-specific EN 50129 [11]. A recurring idea in these documents is the fail-safe principle: when a fault is detected, the system must move to a defined safe state rather than continue as if nothing had happened. The monitor designed in this project follows that idea in a simplified, educational form; it is not a safety-certified product.

2.2 SEQUENCE MONITORING AND WATCHDOG TIMERS

A watchdog is a circuit that supervises another function and raises an alarm when the expected behaviour does not occur within a given time. A sequence monitor combines two checks: an order check, which confirms that events arrive in the expected sequence, and a time check, which confirms that each stage finishes before a limit. Neither check can be done by purely combinational logic, because the circuit must remember which stage of the sequence it is in and must count the time spent there. In this project the memory is the state register of a finite state machine and the time base is an 8-bit counter that is cleared whenever the sequence moves forward.

2.3 FINITE STATE MACHINES IN DIGITAL CONTROL

Finite state machines are the standard way of describing control behaviour in digital hardware [6], [7]. A Moore or Mealy machine consists of a state register, next-state logic and output logic. In a Moore machine the outputs depend only on the present state, which makes them glitch-free and easy to reason about. The monitor in this report is a Moore machine with six states: its two main outputs, fail_safe and sequence_ok, are decoded from the state register alone, so they can never be asserted together. Keeping

the number of states small reduces the number of flip-flops and the amount of combinational logic, which is directly reflected in silicon area.

2.4 OPEN-SOURCE ASIC DESIGN FLOW AND SKY130

The SkyWater SKY130 PDK, released as an open-source 130 nm process design kit [1], has made it possible for students and small teams to design real, manufacturable chips without proprietary licences. Around it an open tool chain has matured: Icarus Verilog for simulation [5], Yosys for logic synthesis [3], OpenSTA for static timing analysis [4], OpenROAD-based place-and-route, Magic for layout and design-rule checking [8], and OpenLane, which automates the complete RTL-to-GDSII flow [2]. Textbooks on CMOS VLSI design [7] give the background on standard cells, timing and physical design that these tools put into practice.

2.5 SUMMARY AND RESEARCH GAP

Existing level crossing controllers and monitors are mostly relay, PLC or microcontroller based and therefore depend either on large electromechanical installations or on software. Open-source ASIC flows, on the other hand, are usually demonstrated on generic examples such as counters or processors. This project combines the two: it applies a complete, open-source, sign-off-quality ASIC flow to a small application-specific safety monitor that addresses a real railway problem, and it documents timing closure across all PVT corners both before and after layout.

CHAPTER 3
SOFTWARE AND HARDWARE REQUIREMENTS

3.1 SOFTWARE REQUIREMENTS

All design steps were carried out with open-source tools. Table 3.1 lists the tools and the role of each one in the project.

Table 3.1: Software tools used

Tool	Version / source	Purpose in the project
Verilog HDL	IEEE 1364-2005 (Verilog-2005)	Macro-free RTL description of the monitor and its testbench
Icarus Verilog (iverilog, vvp)	10.1	RTL and gate-level simulation
GTKWave / VCD	--	Viewing simulation waveforms
Yosys	0.29	Logic synthesis and technology mapping to sky130_fd_sc_hd
OpenSTA	2.4.0	Static timing analysis over 16 PVT corners, pre- and post-layout
OpenLane	v0.21 (Docker)	Automated RTL-to-GDSII flow: floorplan, placement, CTS, routing
OpenROAD (inside OpenLane)	--	Placement, resizer, CTS, global and detailed routing (TritonRoute)
Magic	--	Layout extraction, DRC and GDSII generation
KLayout	--	GDS viewing and XOR check
LVS and CVC checkers	Part of OpenLane	Layout-versus-schematic and circuit-validity checks
SKY130A PDK	sky130_fd_sc_hd library	Standard cells, liberty timing models, LEF and tech files

3.2 HARDWARE REQUIREMENTS

The deliverable of this project is an ASIC layout, so the principal "hardware" is the target technology and the silicon die itself. The monitor is intended to sit in a level crossing control cabinet as a parallel safety checker, between the crossing sensors and the alarm and signalling equipment. Table 3.2 summarizes the technology and the external elements, and explains why each is relevant.

Table 3.2: Hardware and technology elements

Element	Specification	Relevance to the project
Process technology	SkyWater SKY130, 130 nm, sky130A	Open PDK that allows a real, manufacturable layout
Standard-cell library	sky130_fd_sc_hd (high density)	Provides the logic gates, multiplexers and D flip-flops used by synthesis
Die / core area	100 µm × 100 µm die; core 88.96 µm × 78.24 µm	Fixed die size of the project constraint
Clock	20 MHz (50 ns period)	Single synchronous clock for the state, counter and error registers
Train detection sensors (external)	Digital signals train_approach and train_depart	Tell the monitor that a train is approaching and that it has cleared the crossing
Crossing feedback sensors (external)	Digital signals light_on_sensor, barrier_down_sensor, barrier_up_sensor	Confirm that the lights are on and that the barrier is fully down or fully up
Alarm / signalling interface (external)	fail_safe output	Drives the alarm or tells the signalling system that the crossing is unsafe
Indicators (external)	sequence_ok, error_code[2:0]	Status lamp and a 3-bit diagnostic display or logger

3.3 DESIGN CONSTRAINTS

The only timing constraint is a 20 MHz clock with 0.1 ns clock uncertainty, written in the SDC file as create_clock -name clk -period 50 and set_clock_uncertainty 0.1. The RTL must be synthesizable, macro-free Verilog-2005: no `define, `include or `ifdef, and all constants are declared with localparam. The physical constraints are a 100 µm × 100 µm die with an absolute floorplan, a core utilisation target of 30 %, a placement target density of 0.40 and basic placement, as discussed in Section 4.6. The pins are arranged by a pin-order file: the clock, reset and the five sensor inputs on the north edge and the five outputs (fail_safe, sequence_ok and error_code[2:0]) on the south edge. The design therefore has seven input bits and five output bits, twelve I/O pins in total.

CHAPTER 4
METHODOLOGY

4.1 SYSTEM OVERVIEW

The monitor receives five sensor inputs, a clock and a synchronous reset. The two train sensors report that a train is approaching and that it has departed, and the three feedback sensors report that the lights are on, that the barrier is down and that the barrier is up. The monitor produces three outputs: fail_safe, which is 1 when a sequence violation or timeout has been detected; sequence_ok, which is 1 while a crossing sequence is proceeding normally; and a 3-bit error_code that tells which violation occurred. Table 4.1 gives the functional specification and Table 4.2 the interface.

Table 4.1: Functional specification

Condition	Input event	Action of the monitor
Idle, no train	train_approach = 0	Stay in S_IDLE; sequence_ok = 0, fail_safe = 0
Train approaching	train_approach = 1	Move to S_WARN (lights should turn ON)
Lights ON in time	light_on_sensor = 1 before the S_WARN limit	Move to S_BARRIER (barrier should go down)
Barrier DOWN in time	barrier_down_sensor = 1 before the S_BARRIER limit	Move to S_PASSING (train is crossing)
Train departed	train_depart = 1 before the S_PASSING limit	Move to S_CLEAR (barrier should go up)
Barrier UP in time	barrier_up_sensor = 1 before the S_CLEAR limit	Return to S_IDLE (safe state)
Timeout in a timed state	Counter reaches the limit of the state	Move to S_FAULT; fail_safe = 1; timeout code latched
Out-of-order sensor	For example barrier_down_sensor before lights ON, or train_depart in S_IDLE or S_WARN	Move to S_FAULT; fail_safe = 1; error code latched
reset = 1	any	S_IDLE; counter, error register and outputs cleared

Table 4.2: Interface of the monitor

Signal	Direction	Width	Description
clk	Input	1	System clock, rising-edge active (20 MHz)
reset	Input	1	Synchronous reset, active high
train_approach	Input	1	Sensor: a train is approaching the crossing
train_depart	Input	1	Sensor: the train has cleared the crossing
light_on_sensor	Input	1	Feedback: the warning lights are ON
barrier_down_sensor	Input	1	Feedback: the barrier is fully DOWN
barrier_up_sensor	Input	1	Feedback: the barrier is fully UP
fail_safe	Output	1	1 = sequence violation or timeout; the crossing is unsafe
sequence_ok	Output	1	1 = the sequence is proceeding normally
error_code	Output	3	000 = no error; other values identify the violation (Table 4.4)

The design therefore has 7 input bits and 5 output bits (12 I/O pins). The monitor does not drive the barriers or the lights; it is a parallel safety checker. An interlock is built into the output decoding: fail_safe and sequence_ok are never asserted together. Each timed state has its own limit in clock cycles, listed in Table 4.3, and the reason for a failure is latched in the 3-bit error register using the codes of Table 4.4. Figure 4.1 shows the timing of a normal safe sequence and of a fault case.

Table 4.3: Timeout limits of the timed states

State	Limit (cycles)	Time at 20 MHz	Rationale
S_WARN	200	10.00 µs	Lights should respond quickly
S_BARRIER	250	12.50 µs	Barrier descent has a mechanical delay
S_PASSING	255 (maximum of an 8-bit counter)	12.75 µs	Longest legitimate train transit
S_CLEAR	200	10.00 µs	Barrier ascent should take a similar time
S_IDLE, S_FAULT	none	--	S_IDLE waits for a train; S_FAULT is latched until reset

Table 4.4: Error codes

Code	Meaning	Where it is detected
000	No error	--
001	Timeout: the barrier did not go down in time	S_BARRIER
010	barrier_down_sensor asserted before the lights were ON	S_WARN
011	train_depart asserted before the barrier was down	S_IDLE or S_WARN
100	barrier_down_sensor asserted while no train was expected	S_IDLE
101	Timeout: the barrier did not go up in time	S_CLEAR
110	Timeout: the lights did not turn ON in time	S_WARN
111	Timeout: the train did not depart within the limit (listed as reserved in the original specification, used by the RTL for S_PASSING)	S_PASSING

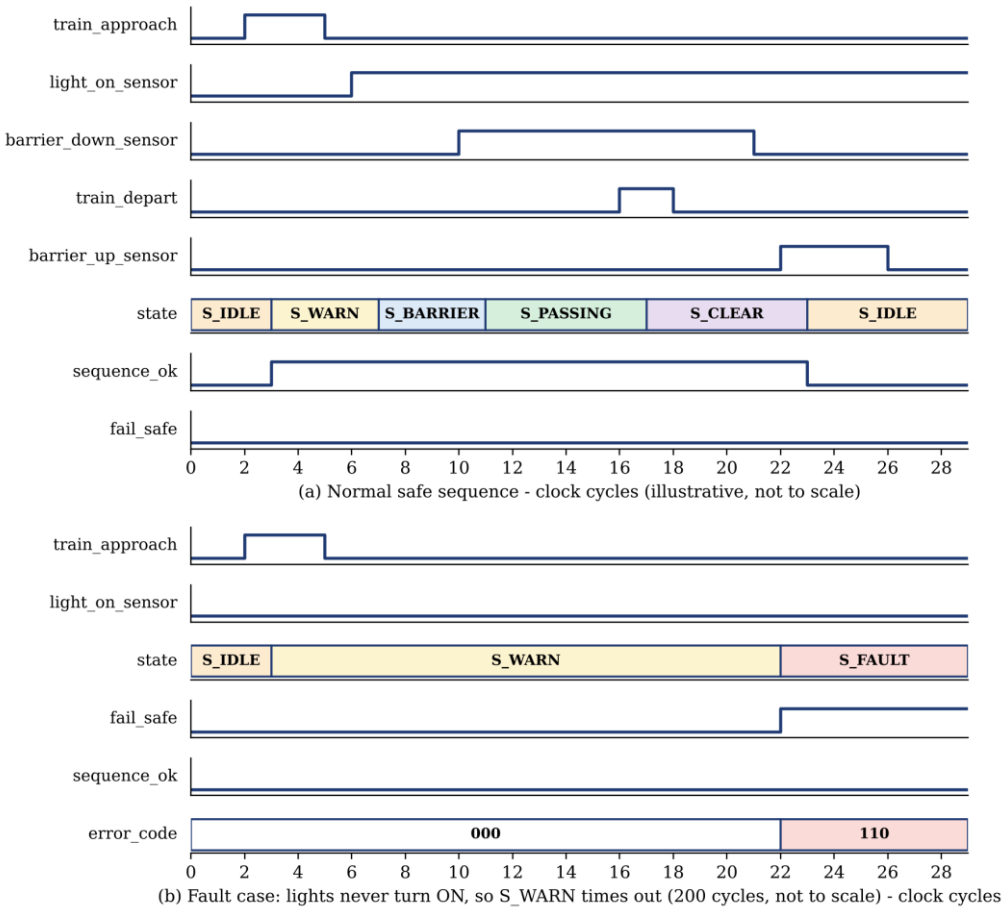


Figure 4.1: Timing of a normal safe sequence and of a fault case (illustrative)

4.2 BLOCK DIAGRAM

Figure 4.2 shows the block diagram. The sequence checker decodes the five sensor inputs and decides whether the event that has just arrived is the one expected in the present state, an out-of-order event, or no event. The timeout limit comparators compare the shared 8-bit counter with the limit of the present state. The next-state logic combines these results with the present state to produce the next state and the error code to be latched, and the reset logic clears the state, counter and error registers. The storage elements are the state register, the 8-bit timeout counter and the 3-bit error register. The output decoder produces fail_safe, sequence_ok and error_code from the registers alone.

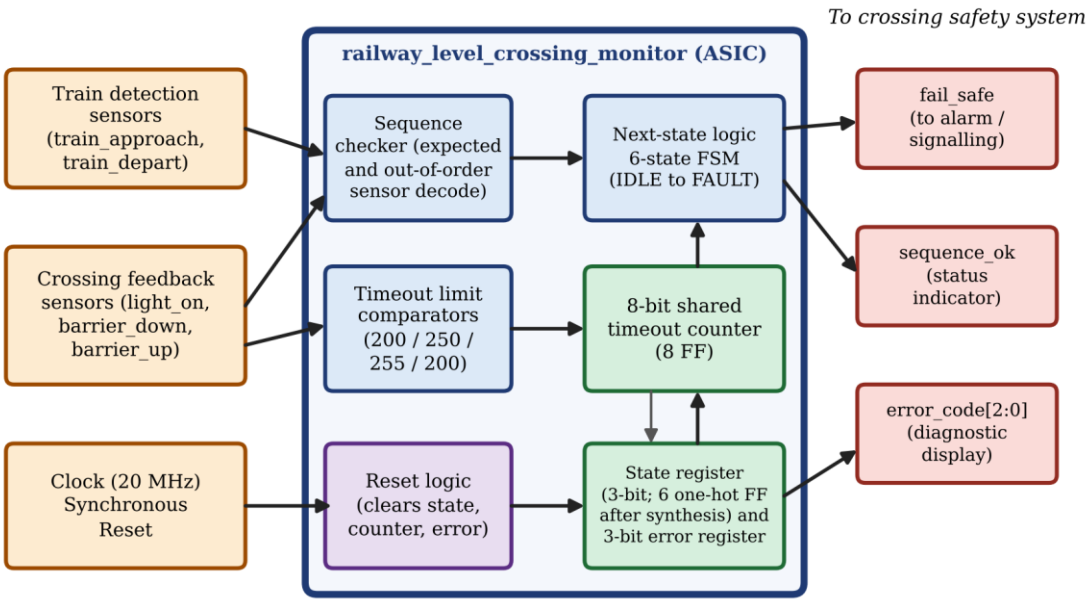


Figure 4.2: Block diagram of the Railway Level Crossing Monitor

4.3 FLOWCHART

The decision order implemented in the RTL is shown in Figure 4.3. Reset has the highest priority. A latched fault is held until reset. In every other state the expected sensor event is tested first, then an out-of-order event (only in S_IDLE and S_WARN), and finally the timeout; if none of them applies the counter is incremented and the machine stays where it is.

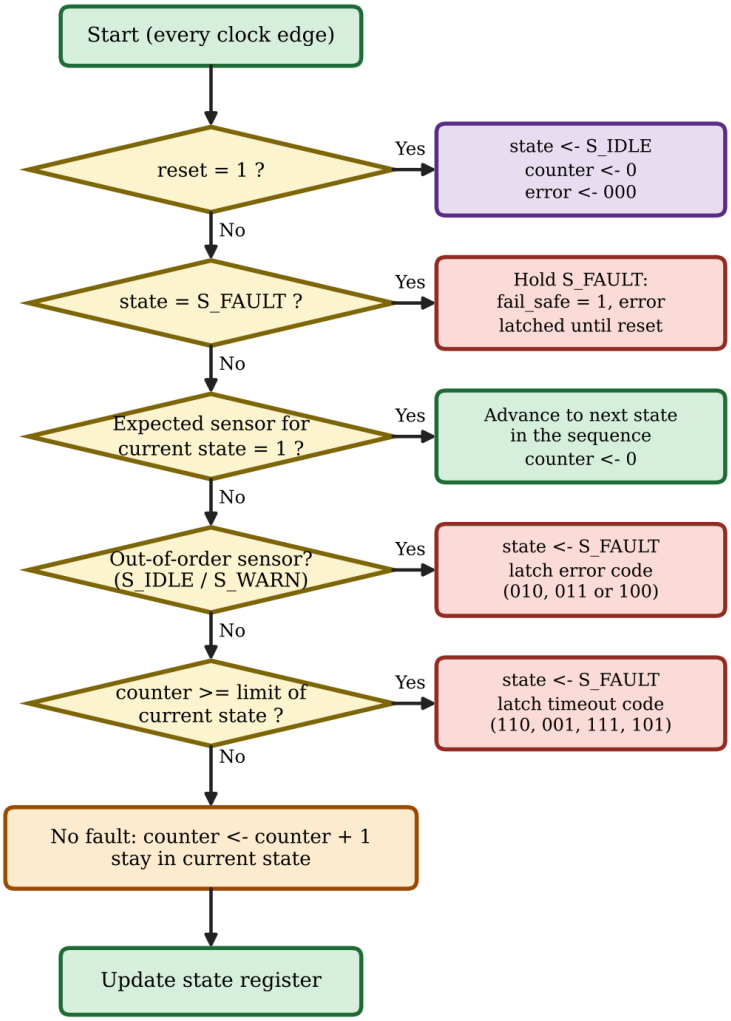


Figure 4.3: Flowchart of the monitor operation

4.4 ALGORITHM

On every rising clock edge the monitor executes the following algorithm.

1. If reset = 1, load S_IDLE into the state register and clear the timeout counter and the error register.
2. Otherwise evaluate the present state. In S_IDLE the counter is held at 0; if train_approach = 1 go to S_WARN, else if barrier_down_sensor = 1 go to S_FAULT with code 100, else if train_depart = 1 go to S_FAULT with code 011.
3. In S_WARN: if light_on_sensor = 1 go to S_BARRIER and clear the counter; else if barrier_down_sensor = 1 go to S_FAULT with code 010; else if train_depart = 1 go to S_FAULT with code 011; else if counter >= 200 go to S_FAULT with code 110; otherwise increment the counter.

4. In S_BARRIER: if barrier_down_sensor = 1 go to S_PASSING and clear the counter; else if counter >= 250 go to S_FAULT with code 001; otherwise increment the counter.
5. In S_PASSING: if train_depart = 1 go to S_CLEAR and clear the counter; else if counter >= 255 go to S_FAULT with code 111; otherwise increment the counter.
6. In S_CLEAR: if barrier_up_sensor = 1 return to S_IDLE and clear the counter; else if counter >= 200 go to S_FAULT with code 101; otherwise increment the counter.
7. In S_FAULT: stay in S_FAULT, hold the counter at 0 and keep the error code latched. Only reset leaves this state.
8. Decode the outputs from the state register: S_FAULT gives fail_safe = 1 and error_code equal to the latched code; S_IDLE gives all outputs 0; every other state gives sequence_ok = 1.

4.5 SYSTEM ARCHITECTURE

4.5.1 Finite state machine

The monitor is a six-state Moore machine, shown in Figure 4.4. S_IDLE (000) is the reset state and means that no train is expected. S_WARN (001) waits for the lights to come on, S_BARRIER (010) waits for the barrier to reach the down position, S_PASSING (011) represents the train crossing the road and S_CLEAR (100) waits for the barrier to go up again. S_FAULT (101) is entered on any violation or timeout and is left only by reset. The two unused state codes (110 and 111) return to S_IDLE through the default branch of the case statement, so the machine can never remain in an undefined state. In the RTL the state is a 3-bit binary code; during synthesis Yosys re-encoded it as one-hot, so the synthesized netlist uses six state flip-flops.

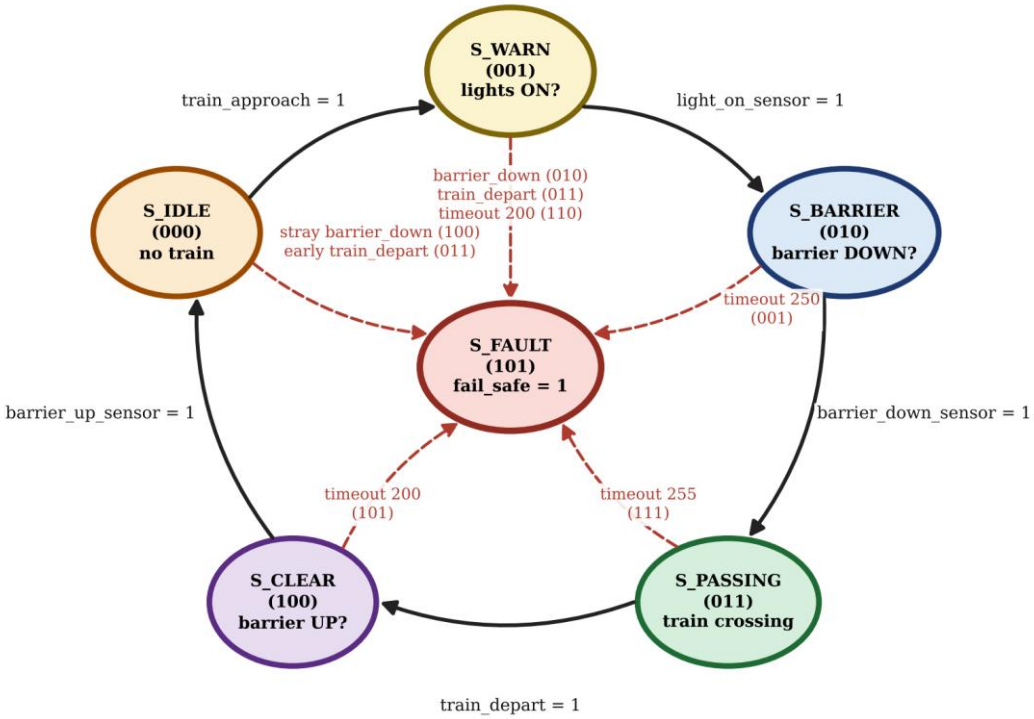


Figure 4.4: State diagram of the monitor

4.5.2 Sequential and combinational logic

The sequential part consists of the state register, the 8-bit timeout counter and the 3-bit error register, all with synchronous reset: 14 flip-flops in the RTL, which become 17 in the synthesized netlist (6 one-hot state flip-flops, 8 counter flip-flops and 3 error flip-flops) because of the one-hot re-encoding. The counter is shared by all timed states and is cleared on every transition, so a single counter is enough to time the four stages. The combinational part consists of the output decoder, which is written in a separate always @(*) block with safe default values so that no latches are inferred, and of the next-state, comparator and error-latching logic inside the clocked block. There are no memories and no latches. The complete RTL listing is given in Appendix A.

Because fail_safe and sequence_ok are decoded from the state register only, they cannot be asserted together and they cannot glitch because of a changing sensor input. The error register keeps its value while the machine is in S_FAULT, and error_code is shown at the output only in S_FAULT; in every other state it reads 000.

4.6 DESIGN AND VERIFICATION FLOW

The design was carried through the following steps: (1) RTL design and self-checking simulation; (2) Yosys synthesis to the sky130_fd_sc_hd library; (3) pre-layout STA in 16 PVT corners; (4) gate-level simulation with the SKY130 cell models; (5) OpenLane physical design including floorplan, placement, CTS, routing, DRC, LVS, antenna and CVC checks; (6) post-layout STA with extracted parasitics (SPEF) in the same 16 corners; and (7) comparison of pre- and post-layout timing. RTL, gate-level and physical-design results are reported separately in Chapter 5. Table 4.5 lists the main design decisions and the reasons behind them.

Table 4.5: Design decisions and trade-offs

Decision	Reason
6-state FSM	Sufficient to model the safety sequence without over-complicating the design
3-bit binary state in RTL (one-hot after synthesis)	Compact, readable RTL; Yosys chose one-hot encoding automatically, giving 6 state flip-flops
8-bit shared timeout counter	One counter reused by all timed states saves area; it is cleared on every transition
Latched fault, cleared only by reset	Fail-safe behaviour: a violation cannot disappear by itself
Monitor only, no direct drive of barriers or lights	It acts as a parallel safety checker and cannot interfere with the normal crossing control
Functional flip-flop model for GLS	The SKY130 UDP hold check produces false X values; timing is verified by STA instead
FP_CORE_UTIL = 30	The design occupies about 19 % of the core; leaves room for routing
PL_TARGET_DENSITY = 0.40	Above the actual utilisation; prevents RePlAce divergence
PL_BASIC_PLACEMENT = 1	Robust for a small design in a comparatively large die
50 ns clock period	The worst slow-corner path is about 20 ns; a 50 ns period gives large timing margin and easy closure

CHAPTER 5
RESULTS AND DISCUSSION

5.1 RTL SIMULATION

A self-checking testbench with 13 test cases was written and simulated with Icarus Verilog. Each case applies a sensor condition and checks the state and the outputs. Test 12 contains two checks (12a and 12b), so the run reports 14 checks, and all 14 passed. The test cases and expected results are listed in Table 5.1.

Table 5.1: RTL test cases and results

Test	Stimulus	Expected result	Result
1	reset = 1	S_IDLE; fail_safe, sequence_ok and error_code all 0	PASS
2	Full correct sequence completed	Back in S_IDLE, error_code = 000	PASS
3	train_approach = 1	S_WARN, sequence_ok = 1, fail_safe = 0	PASS
4	light_on_sensor = 1 in time	S_BARRIER, sequence_ok = 1	PASS
5	barrier_down_sensor = 1 in time	S_PASSING, sequence_ok = 1	PASS
6	train_depart = 1	S_CLEAR, sequence_ok = 1	PASS
7	barrier_up_sensor = 1 in time	S_IDLE, sequence_ok = 0, fail_safe = 0	PASS
8	Lights never turn ON (timeout in S_WARN)	S_FAULT, fail_safe = 1, error_code = 110	PASS
9	Barrier never goes DOWN (timeout in S_BARRIER)	S_FAULT, fail_safe = 1, error_code = 001	PASS
10	barrier_down_sensor before lights ON	S_FAULT, fail_safe = 1, error_code = 010	PASS
11	train_depart while in S_WARN	S_FAULT, fail_safe = 1, error_code = 011	PASS
12	Fault latch: 50 cycles in S_FAULT (12a), then reset (12b)	12a: fail_safe stays 1; 12b: reset returns S_IDLE, outputs 0	PASS (2 checks)
13	Interlock check	sequence_ok and fail_safe never both 1	PASS

Figure 5.1 shows the simulation waveform of the testbench. After reset the full safe sequence runs from S_IDLE through S_WARN, S_BARRIER, S_PASSING and S_CLEAR and back to S_IDLE with sequence_ok high and fail_safe low. The waveform then shows the timeout in S_WARN at about 2.2 µs (error_code 110), the timeout in S_BARRIER at about 5.0 µs (error_code 001), the out-of-order barrier_down_sensor (error_code 010), the early train_depart (error_code 011) and the latched fault that is cleared only by reset.

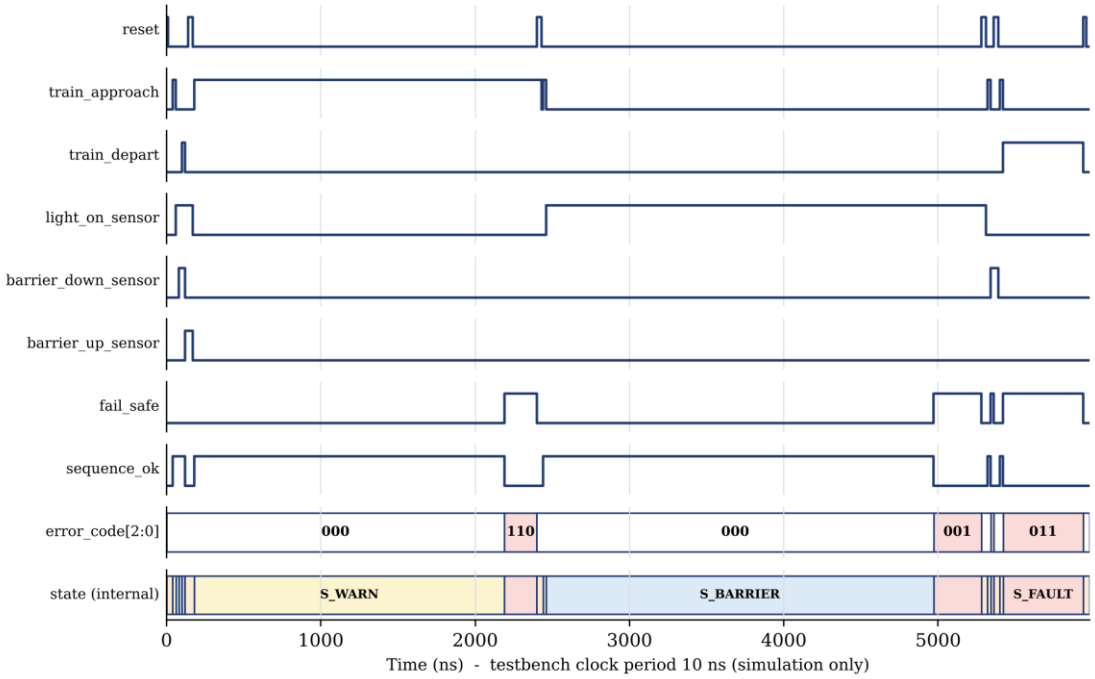


Figure 5.1: RTL simulation waveform of the 13 test cases

5.2 LOGIC SYNTHESIS

Yosys mapped the RTL to the sky130_fd_sc_hd library inside the OpenLane flow. The result is 139 cells with a total cell area of 1,300 µm², containing exactly 17 flip-flops (6 for the one-hot state, 8 for the timeout counter and 3 for the error code), no latches and no memories (Table 5.2). Compared with the core area of about 6,960 µm² (88.96 µm × 78.24 µm), the cells occupy 19 % utilisation.

Table 5.2: Synthesis results

Cell type	Count	Cell type	Count
a2111o_2	1	o211a_2	4
a211o_2	3	o21a_2	4
a21bo_2	1	o21ai_2	3
a21o_2	3	o221a_2	11
a31o_2	1	o221ai_2	5
a41o_2	1	o22a_2	1
and2_2	3	o22ai_2	4
and3_2	1	o311a_2	1
buf_1	12	o31a_2	1
dfxtp_2 (flip-flop)	17	o32a_2	5
inv_2	28	or2_2	6
mux2_1	7	or3_2	2
nand2_2	1	or4_2	1
nor2_2	12		
Total cells	139	Chip area	1,300 µm²

A separate standalone Yosys run with a plain ABC mapping gave a smaller netlist of 122 cells (about 920 µm² computed from the typical-corner liberty file) with the same 17 flip-flops. That netlist was used for gate-level simulation and for the pre-layout STA, while the OpenLane synthesis result of Table 5.2 is the one that was placed and routed. In both cases the flip-flop count is identical and no latches are present.

5.3 PRE-LAYOUT STATIC TIMING ANALYSIS

Timing was analysed with OpenSTA at all 16 SKY130 corners for the 50 ns clock. Every corner meets setup and hold. The worst setup slack is +29.93 ns at ss_n40C_1v28 and the worst hold slack is +0.18 ns at ff_n40C_1v95. In the slowest corner the worst setup path starts at a state flip-flop, passes through six gates and ends at another state flip-flop; its data arrival time is 19.11 ns against a required time of 49.04 ns. The worst hold path is a single flip-flop feeding itself through two gates, with an arrival of 0.258 ns against a required 0.082 ns. The values for all corners are tabulated together with the post-layout results in Table 5.5.

5.4 GATE-LEVEL SIMULATION

The same stimulus was run on the synthesized netlist with the SKY130 cell models. A functional-only flip-flop model (ff_override.v) was used because the SKY130 flip-flop primitive includes a hold-check notifier which, when the Q-to-D feedback arrives inside

the hold window, spuriously produces unknown (X) values in zero-delay simulation. Since timing is signed off by STA in all corners, the functional model is standard practice. During synthesis Yosys re-encoded the 3-bit FSM state as 6-bit one-hot, so the gate-level testbench compares only the primary outputs (fail_safe, sequence_ok and error_code) and not the internal state, which is the usual practice for gate-level checks. With these changes all 14 checks passed (14/14), confirming that the synthesized netlist is functionally equivalent to the RTL.

5.5 PHYSICAL DESIGN

The OpenLane v0.21 flow completed successfully in 3 min 27 s (routing in 2 min 46 s) with a peak memory of about 443 MB. The die is 100 µm × 100 µm (0.01 mm²) and the core is 88.96 µm × 78.24 µm. The final layout contains 139 logic cells, 90 tap cells and 56 end caps (146 physical cells in the flow summary) plus fill cells in the remaining free sites, with a total wire length of 7,135 µm and 1,428 vias. The layout checks are listed in Table 5.3 and the layout views in Figures 5.2 to 5.5.

Table 5.3: Physical verification results

Check	Result
TritonRoute violations	0
Short violations	0
Metal spacing violations	0
Off-grid violations	0
Magic DRC violations	0
Antenna violations	0
LVS errors (no net, device, pin or property mismatches)	0
CVC errors	0

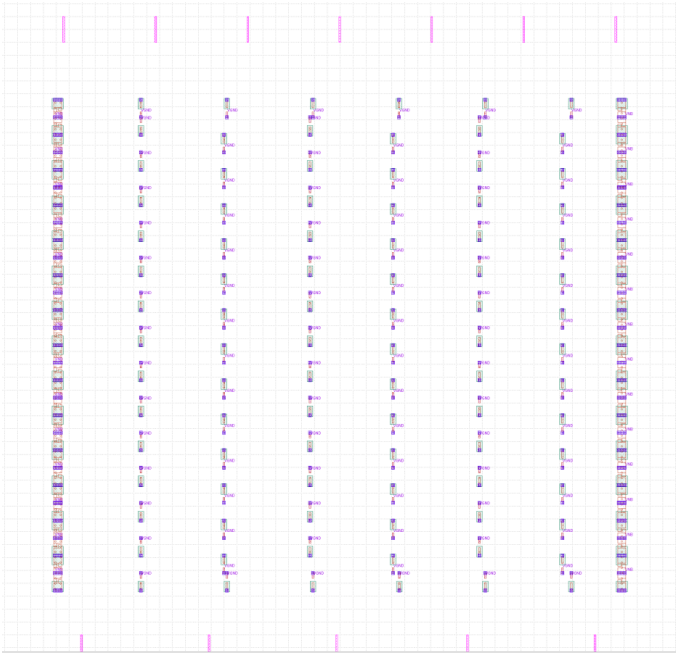


Figure 5.2: Floorplan view (pins and tap cells)

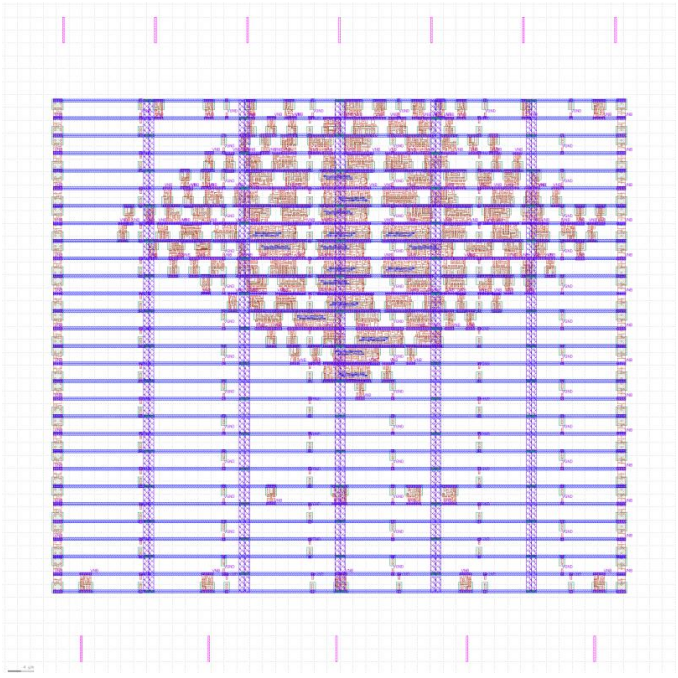


Figure 5.3: Placement view after global placement and resizing

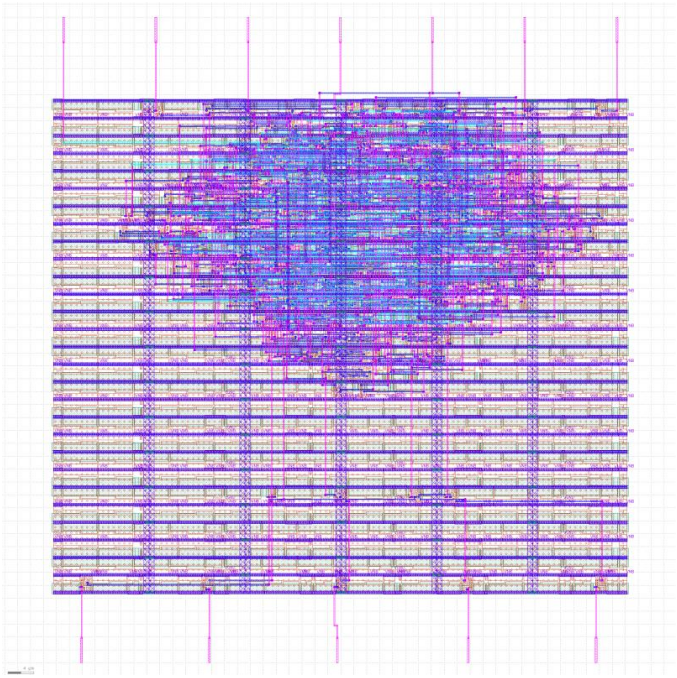


Figure 5.4: Routed layout view

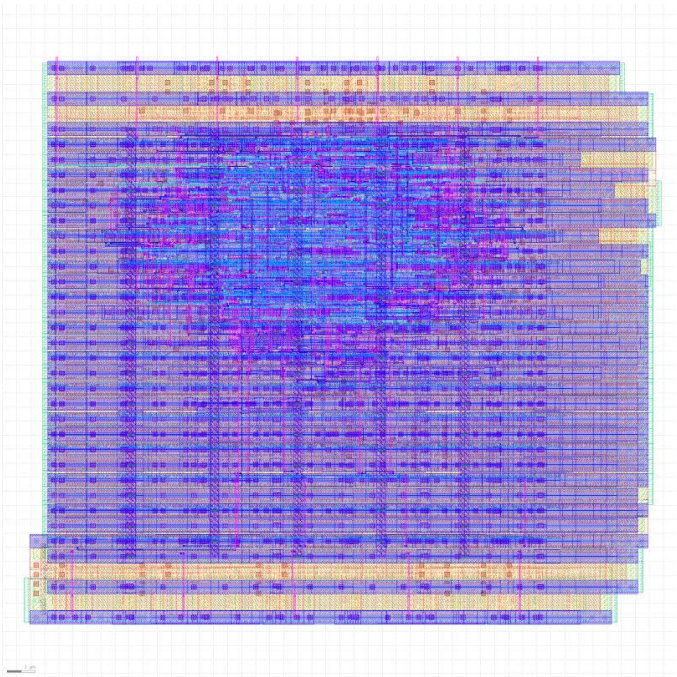


Figure 5.5: Final GDSII layout view

The tool prints the warning that the core is too small for the standard power-distribution pitch (about 153 µm, larger than the 100 µm die); OpenLane then minimises the power grid and the flow still completes with all checks passing. The LVS result proves that the layout netlist is logically identical to the synthesized netlist. The final GDSII stream is 512,482 bytes long and carries a valid stream header.

5.6 POST-LAYOUT TIMING AND PRE/POST COMPARISON

After routing, parasitics were extracted to a SPEF file and STA was repeated in all 16 corners. Setup slack stays above +28.85 ns and hold slack above +0.20 ns in every corner, so all corners pass. Table 5.5 compares pre- and post-layout slacks. The setup slack decreases by between 0.250 ns and 1.080 ns because of the real wire parasitics, and the hold slack increases by between 0.020 ns and 0.740 ns. The largest changes occur in the slowest corner, ss_n40C_1v28, where the pre-layout setup slack of 29.93 ns falls to 28.85 ns; even there the margin is huge compared with the 50 ns period.

Table 5.4: Summary of timing results

Quantity	Pre-layout	Post-layout
Worst setup slack	+29.93 ns (ss_n40C_1v28)	+28.85 ns (ss_n40C_1v28)
Worst hold slack	+0.18 ns (ff_n40C_1v95)	+0.20 ns (ff_n40C_1v95)
Corners analysed	16	16
Corners passing	16	16

Table 5.5: Pre-layout versus post-layout slack at all 16 corners (ns)

Corner	Setup pre	Setup post	Δ	Hold pre	Hold post	Δ
ff_100C_1v65	48.35	48.03	-0.320	0.26	0.30	+0.040
ff_100C_1v95	48.72	48.43	-0.290	0.19	0.21	+0.020
ff_n40C_1v56	47.78	47.49	-0.290	0.33	0.36	+0.030
ff_n40C_1v65	48.10	47.82	-0.280	0.27	0.30	+0.030
ff_n40C_1v76	48.38	48.11	-0.270	0.23	0.25	+0.020
ff_n40C_1v95	48.69	48.44	-0.250	0.18	0.20	+0.020
ss_100C_1v40	43.23	42.35	-0.880	1.16	1.47	+0.310
ss_100C_1v60	45.51	44.88	-0.630	0.80	0.99	+0.190
ss_n40C_1v28	29.93	28.85	-1.080	2.26	3.00	+0.740
ss_n40C_1v35	36.43	35.69	-0.740	1.66	2.21	+0.550
ss_n40C_1v40	39.20	38.60	-0.600	1.38	1.81	+0.430
ss_n40C_1v44	40.81	40.27	-0.540	1.21	1.57	+0.360
ss_n40C_1v60	44.43	44.04	-0.390	0.79	0.99	+0.200
ss_n40C_1v76	46.08	45.77	-0.310	0.59	0.71	+0.120
tt_025C_1v80	47.77	47.48	-0.290	0.36	0.41	+0.050
tt_100C_1v80	47.86	47.50	-0.360	0.37	0.43	+0.060

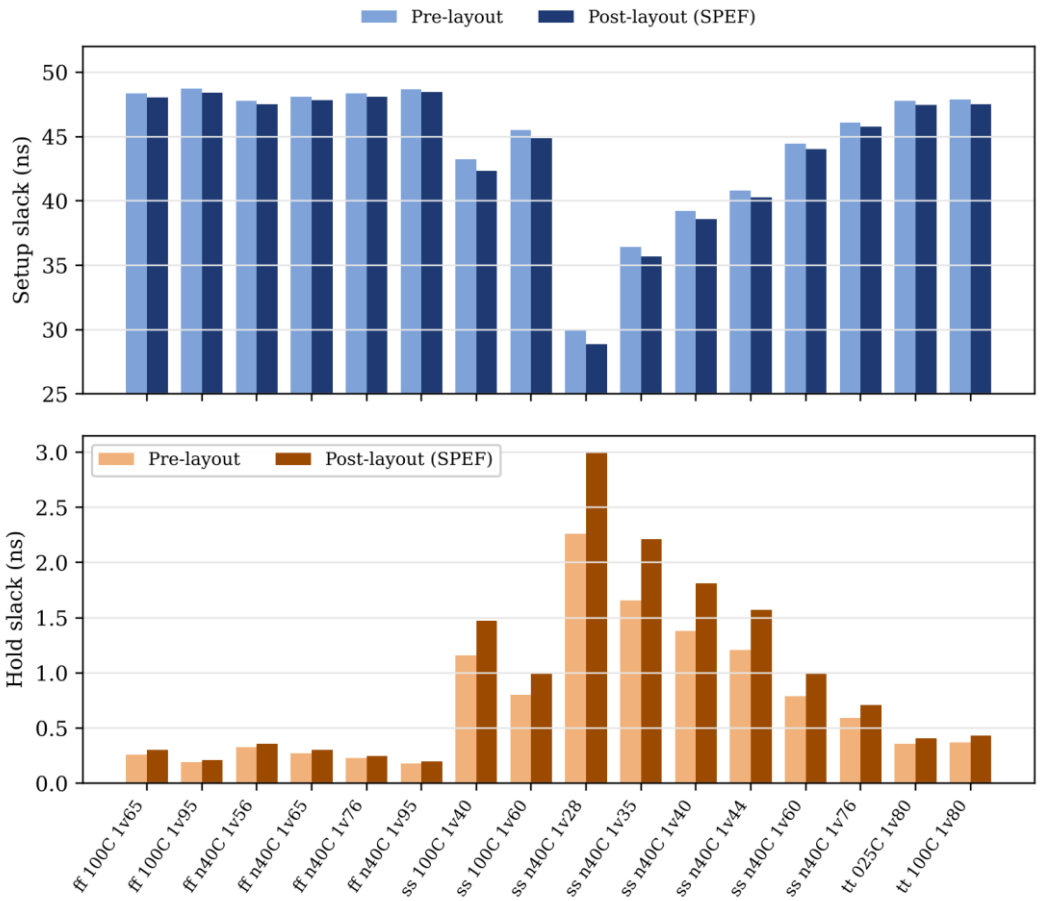


Figure 5.6: Setup and hold slack across the 16 PVT corners, pre- and post-layout

5.7 PROBLEMS ENCOUNTERED AND SOLUTIONS

- **Timeout counter not cleared on state changes:** the first RTL version failed to clear the counter when the state changed, so a stage inherited the time already spent in the previous one. The counter is now cleared on every transition and on reset.
- **Gate-level state mismatch:** Yosys converted the 3-bit binary FSM state into one-hot encoding (6 bits), so the gate-level testbench failed when it compared the 6-bit internal state with the 3-bit RTL constants. The internal-state comparisons were removed and only the primary outputs are verified in gate-level simulation.
- **False X values in gate-level simulation:** the SKY130 flip-flop UDP hold check fired its notifier on the Q-to-D feedback path. A purely functional D flip-flop override (ff_override.v) is used for gate-level simulation, with timing covered by STA.
- **Syntax errors in the PDK functional model:** the open-source sky130_fd_sc_hd.v contained two invalid declarations ("wire 1;" at lines 955

and 1097) that stopped Icarus Verilog from compiling the netlist. The two lines were patched in the local copy.

- **Placement divergence:** the global placer (RePlAce) stopped with the error "[GPL-0302] Filler area is negative" because the actual utilisation (about 20 %) was higher than PL_TARGET_DENSITY (0.10). Increasing PL_TARGET_DENSITY to 0.40 in config.tcl and config.json gave the placer enough room and the flow succeeded.

5.8 COST, USEFULNESS TO SOCIETY AND ENVIRONMENTAL SAFETY

Cost. Silicon cost is roughly proportional to die area. The monitor needs only 1,300 µm² of standard cells in a 0.01 mm² die, and the entire design was done with free, open-source tools and an open PDK, so there are no tool-licence costs. The external parts (sensors, alarm interface, indicators) are inexpensive, widely available components. Because the monitor has no firmware, there are also no software development or update costs. A quantitative cost estimate would depend on the chosen fabrication shuttle or volume and is outside the scope of this report.

Usefulness to society. Level crossing accidents endanger road users and rail passengers and cause serious loss of life and property. A fail-safe, always-on monitor that checks the crossing sequence in real time and raises an immediate alarm when a step is missed or late supports railway signalling engineers and safety-system integrators, and it can run in parallel with the existing crossing controller without interfering with it. The design is small enough to be embedded in a control cabinet, and the open-source design files can be reused for teaching and further development.

Environmental safety. The circuit is a low-complexity CMOS design with very few switching elements, which keeps its dynamic power small (a power analysis was not part of this project). A hardware monitor that never needs a reboot and has no software to update has a long service life with little maintenance. The latched, reset-only fault state ensures that a violation is never silently cleared. The design uses no hazardous materials beyond standard semiconductor manufacturing.

5.9 LIMITATIONS

- The sensor inputs are assumed to be clean, debounced digital signals; there are no input synchronizers or debounce circuits.

- There is no physical barrier-position feedback beyond the barrier_up_sensor and barrier_down_sensor signals, so the monitor is command-and-feedback based only.
- The timeout limits are counted in clock cycles. At the 20 MHz sign-off clock they correspond to 10 to 12.75 µs; in a real crossing the monitor would be clocked from a slow tick or a prescaler would be added so that the limits correspond to seconds.
- The testbench exercises the timeouts of S_WARN and S_BARRIER and two out-of-order cases; the timeouts of S_PASSING and S_CLEAR and the stray barrier_down_sensor and early train_depart cases in S_IDLE are not tested directly.
- The monitor is a single-channel design without redundancy and has not been assessed against the safety-integrity requirements of IEC 61508 or EN 50129; it is an educational design.
- No power estimate was made; the chip was verified in simulation and by layout checks, not fabricated and measured.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 CONCLUSION

A Railway Level Crossing Monitor was specified, designed and implemented as a digital ASIC on the SkyWater SKY130 process. The six-state FSM with an 8-bit shared timeout counter and a 3-bit error register checks that the crossing sequence (lights ON, barrier DOWN, train passes, barrier UP) happens in the correct order and in time, latches any violation in S_FAULT until reset, reports the reason through a 3-bit error code and guarantees that fail_safe and sequence_ok are never asserted together. RTL and gate-level simulations passed all 14 checks of the 13 test cases, synthesis produced 139 cells of 1,300 μm² with 17 flip-flops and no latches, and static timing analysis in 16 PVT corners, before and after layout, showed positive slack everywhere (post-layout worst setup +28.85 ns, worst hold +0.20 ns). The OpenLane flow yielded a 100 μm × 100 μm GDSII layout with zero DRC, LVS, antenna, routing and CVC violations.

The project demonstrates a complete, reproducible, open-source RTL-to-GDSII flow and shows that a small dedicated monitor can replace a microcontroller for this task with deterministic and fail-safe behaviour.

6.2 FUTURE SCOPE

- Add input synchronizers and debounce logic for the sensor inputs, and a slow-tick prescaler so that the timeout limits correspond to seconds.
- Make the timeout limits programmable through configuration registers so that crossings with different barrier speeds can use different limits.
- Extend the testbench to cover the S_PASSING and S_CLEAR timeouts and every out-of-order case, and add formal checks of the interlock and latch properties.
- Add a redundant (dual-channel) monitor with output comparison for higher safety integrity.
- Add a fault log or serial interface that reports the error code and the time of the fault.
- Perform power analysis, and submit the design to an open-source shuttle for fabrication and measurement.

REFERENCES

[1] SkyWater Technology Foundation and Google, 'SkyWater Open Source PDK (SKY130)', Documentation, https://skywater-pdk.readthedocs.io.

[2] M. Shalan and T. Edwards, 'Building OpenLANE: A 130nm OpenROAD-based Tapeout-Proven Flow', Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD), 2020.

[3] C. Wolf and J. Glaser, 'Yosys -- A Free Verilog Synthesis Suite', Proc. 21st Austrian Workshop on Microelectronics (Austrochip), 2013.

[4] The OpenROAD Project, 'OpenSTA: Gate-level static timing verifier', https://github.com/The-OpenROAD-Project/OpenSTA.

[5] S. Williams, 'Icarus Verilog', open-source Verilog simulation and synthesis tool, http://iverilog.icarus.com.

[6] S. Palnitkar, 'Verilog HDL: A Guide to Digital Design and Synthesis', 2nd ed., Prentice Hall, 2003.

[7] N. H. E. Weste and D. M. Harris, 'CMOS VLSI Design: A Circuits and Systems Perspective', 4th ed., Addison-Wesley, 2011.

[8] J. K. Ousterhout et al., 'Magic: A VLSI Layout System', Proc. 21st Design Automation Conference (DAC), 1984, pp. 152-159.

[9] IEEE Standard for Verilog Hardware Description Language, IEEE Std 1364-2005, 2006.

[10] International Electrotechnical Commission, 'IEC 61508: Functional safety of electrical/electronic/programmable electronic safety-related systems'.

[11] CENELEC, 'EN 50129: Railway applications - Communication, signalling and processing systems - Safety related electronic systems for signalling'.

APPENDICES

APPENDIX A: RTL SOURCE CODE

```
// ============================================================
// Railway Level Crossing Monitor
// SKY130 RTL-to-GDSII ASIC Design Project
//
// Verifies the safe sequence of a level crossing and raises a
// fail-safe alarm on violation or timeout.
//
// Macro-free Verilog-2005. Constants defined via localparam.
// ============================================================

module railway_level_crossing_monitor (
    input  wire       clk,
    input  wire       reset,
    input  wire       train_approach,
    input  wire       train_depart,
    input  wire       light_on_sensor,
    input  wire       barrier_down_sensor,
    input  wire       barrier_up_sensor,
    output reg        fail_safe,
    output reg        sequence_ok,
    output reg  [2:0] error_code
);

    // ------------------------------------------------------------
    // FSM state encoding (3-bit binary)
    // ------------------------------------------------------------
    localparam [2:0] S_IDLE    = 3'b000;
    localparam [2:0] S_WARN    = 3'b001;
    localparam [2:0] S_BARRIER = 3'b010;
    localparam [2:0] S_PASSING = 3'b011;
    localparam [2:0] S_CLEAR   = 3'b100;
    localparam [2:0] S_FAULT   = 3'b101;

    // ------------------------------------------------------------
    // Error codes
    // ------------------------------------------------------------
    localparam [2:0] ERR_NONE            = 3'b000;
    localparam [2:0] ERR_TIMEOUT_BARRIER = 3'b001;
    localparam [2:0] ERR_BARRIER_EARLY   = 3'b010;
    localparam [2:0] ERR_DEPART_EARLY    = 3'b011;
    localparam [2:0] ERR_BARRIER_STRAY   = 3'b100;
    localparam [2:0] ERR_TIMEOUT_CLEAR   = 3'b101;
    localparam [2:0] ERR_TIMEOUT_WARN    = 3'b110;
    localparam [2:0] ERR_TIMEOUT_PASSING = 3'b111;

    // ------------------------------------------------------------
    // Timeout limits (in clk cycles)
    // ------------------------------------------------------------
    localparam [7:0] LIMIT_WARN    = 8'd200;
    localparam [7:0] LIMIT_BARRIER = 8'd250;
    localparam [7:0] LIMIT_PASSING = 8'd255;
    localparam [7:0] LIMIT_CLEAR   = 8'd200;

    // ------------------------------------------------------------
    // Sequential elements (14 flip-flops total)
    // ------------------------------------------------------------
    reg [2:0] state;         // FSM state register
    reg [7:0] counter;       // shared timeout counter
    reg [2:0] error_code_r;  // latched error code

    // ------------------------------------------------------------
    // FSM + counter + error register
    // ------------------------------------------------------------
    always @(posedge clk) begin
        if (reset) begin
            state        <= S_IDLE;
            counter      <= 8'd0;
            error_code_r <= ERR_NONE;
        end
        else begin
```

```
case (state)

    // -------- S_IDLE: no train, all clear --------
    S_IDLE: begin
        counter <= 8'd0;
        if (train_approach) begin
            state <= S_WARN;
        end
        else if (barrier_down_sensor) begin
            state        <= S_FAULT;
            error_code_r <= ERR_BARRIER_STRAY;
        end
        else if (train_depart) begin
            state        <= S_FAULT;
            error_code_r <= ERR_DEPART_EARLY;
        end
    end

    // -------- S_WARN: lights should turn ON --------
    S_WARN: begin
        if (light_on_sensor) begin
            state   <= S_BARRIER;
            counter <= 8'd0;
        end
        else if (barrier_down_sensor) begin
            state        <= S_FAULT;
            error_code_r <= ERR_BARRIER_EARLY;
        end
        else if (train_depart) begin
            state        <= S_FAULT;
            error_code_r <= ERR_DEPART_EARLY;
        end
        else if (counter >= LIMIT_WARN) begin
            state        <= S_FAULT;
            error_code_r <= ERR_TIMEOUT_WARN;
        end
        else begin
            counter <= counter + 8'd1;
        end
    end

    // -------- S_BARRIER: barrier should go DOWN --------
    S_BARRIER: begin
        if (barrier_down_sensor) begin
            state   <= S_PASSING;
            counter <= 8'd0;
        end
        else if (counter >= LIMIT_BARRIER) begin
            state        <= S_FAULT;
            error_code_r <= ERR_TIMEOUT_BARRIER;
        end
        else begin
            counter <= counter + 8'd1;
        end
    end

    // -------- S_PASSING: train is crossing --------
    S_PASSING: begin
        if (train_depart) begin
            state   <= S_CLEAR;
            counter <= 8'd0;
        end
        else if (counter >= LIMIT_PASSING) begin
            state        <= S_FAULT;
            error_code_r <= ERR_TIMEOUT_PASSING;
        end
        else begin
            counter <= counter + 8'd1;
        end
    end

    // -------- S_CLEAR: barrier should go UP --------
    S_CLEAR: begin
        if (barrier_up_sensor) begin
            state   <= S_IDLE;
            counter <= 8'd0;
        end
        else if (counter >= LIMIT_CLEAR) begin
```

```
                        state        <= S_FAULT;
                        error_code_r <= ERR_TIMEOUT_CLEAR;
                    end
                    else begin
                        counter <= counter + 8'd1;
                    end
                end

                // -------- S_FAULT: latched until reset --------
                S_FAULT: begin
                    counter <= 8'd0;
                end

                // -------- Safety default --------
                default: begin
                    state        <= S_IDLE;
                    counter      <= 8'd0;
                    error_code_r <= ERR_NONE;
                end

            endcase
        end
    end

    // ------------------------------------------------------------
    // Combinational outputs
    // ------------------------------------------------------------
    always @(*) begin
        // Safe defaults
        fail_safe   = 1'b0;
        sequence_ok = 1'b0;
        error_code  = ERR_NONE;

        if (state == S_FAULT) begin
            fail_safe   = 1'b1;
            sequence_ok = 1'b0;
            error_code  = error_code_r;
        end
        else if (state == S_IDLE) begin
            fail_safe   = 1'b0;
            sequence_ok = 1'b0;
            error_code  = ERR_NONE;
        end
        else begin
            // S_WARN, S_BARRIER, S_PASSING, S_CLEAR
            fail_safe   = 1'b0;
            sequence_ok = 1'b1;
            error_code  = ERR_NONE;
        end
    end

endmodule
```

APPENDIX B: SDC AND OPENLANE CONFIGURATION

```
create_clock -name clk -period 50 [get_ports clk]
set_clock_uncertainty 0.1 [get_clocks clk]


set ::env(PDK_ROOT) "/pdks"
set ::env(PDK) "sky130A"
set ::env(STD_CELL_LIBRARY) "sky130_fd_sc_hd"

set ::env(DESIGN_NAME) "railway_level_crossing_monitor"
set ::env(VERILOG_FILES) "/project/rtl/railway_level_crossing_monitor.v"
set ::env(FP_SIZING) "absolute"
set ::env(DIE_AREA) "0 0 100 100"
set ::env(FP_ASPECT_RATIO) "1.0"
set ::env(FP_CORE_UTIL) "30"
set ::env(PL_TARGET_DENSITY) "0.40"
set ::env(PL_BASIC_PLACEMENT) "1"
set ::env(FP_PIN_ORDER_CFG) "/project/openlane/pin_order.cfg"
set ::env(CLOCK_PORT) "clk"
set ::env(CLOCK_PERIOD) "50.0"
set ::env(DESIGN_IS_CORE) "true"
set ::env(RUN_CTS) "true"
```

```
# pin_order.cfg
#N
clk
reset
train_approach
train_depart
light_on_sensor
barrier_down_sensor
barrier_up_sensor
#S
fail_safe
sequence_ok
error_code\[2\]
error_code\[1\]
error_code\[0\]
```

APPENDIX C: COMMANDS TO REPRODUCE THE FLOW

1. RTL simulation: iverilog -g2012 -o tb.vvp rtl/*.v tb/*.v && vvp tb.vvp
2. Synthesis: yosys -l synth.log synth.ys
3. Pre-layout STA: OpenSTA with the SDC file over 16 SKY130 corners
4. Gate-level simulation: iverilog -g2012 with the functional flip-flop override (gls/ff_override.v)
5. Physical design: OpenLane v0.21 (Docker), DIE_AREA = 0 0 100 100, PDK = sky130A
6. Post-layout STA: OpenSTA with SPEF over 16 SKY130 corners

APPENDIX D: PROJECT DELIVERABLES

Table D.1: Project deliverables

Artifact	File
GDSII (KLayout)	results/railway_level_crossing_monitor.gds
GDSII (Magic)	results/railway_level_crossing_monitor.magic.gds
Parasitics	results/railway_level_crossing_monitor.spef
Routed netlist	results/railway_level_crossing_monitor.synthesis_preroute.v
Flow summary	results/final_summary_report.csv
Pre-/post-layout STA	timing/prelayout, timing/postlayout, timing/comparison.txt
RTL and testbench	rtl/railway_level_crossing_monitor.v, tb/railway_level_crossing_monitor_tb.v
Gate-level simulation files	gls/